Blind Haar-Wavelet Image Watermarking in MATLAB: Interface, Recovery, and Experimental Evaluation

Samuel Eko
University of North Texas
Email: SamuelEko@my.unt.edu

Abstract

This paper presents a MATLAB application for embedding a grayscale image watermark into a color cover image using local 2×2 Haar transforms. The interactive system supports image selection, watermark preparation, adjustable embedding strength, three repetition modes, capacity checking, blind extraction, and lossless Portable Network Graphics (PNG) output. A repeated header stores the parameters needed for extraction, while bit repetition and channel voting support recovery. The application reports cover distortion using peak signal-to-noise ratio (PSNR) and a block-based structural similarity (SSIM) variant, and evaluates recovered data using bit error rate (BER), normalized cross-correlation (NCC), and a checksum. In the documented example, a 32×32 prepared watermark was recovered exactly from an unchanged 1152×768 cover after embedding at strength 16 in Balanced mode. The interface reported 44.29 dB PSNR, 0.9966 block SSIM, 0% BER, and NCC of 1.00000. A separate robustness study considers JPEG compression, cropping, and resizing. The visible evidence does not establish attack-specific robustness for every transformation. The results demonstrate exact recovery of the prepared grayscale watermark in the shown unchanged-image example.

I. Introduction

Digital image watermarking embeds information into a host image while attempting to preserve its visual usefulness. A practical watermarking system must address at least two different requirements: the host should remain visually similar to the original, and the embedded message should be recoverable with a clearly defined success criterion. These requirements can conflict when the watermark is made stronger or repeated more often.

A MATLAB application hides one image in another by modifying local Haar detail coefficients. The user selects two images, watermark size, and strength before a capacity-checked embedding step. The interface displays processing stages and metrics, and an Extract tab reads watermarked PNG files without the original cover.

We examine exact recovery and measured cover quality in the unchanged case, then assess what the available compression and geometric-change records support. Several robustness-study labels are truncated; no unreported experiments are asserted.

II. Related Work

Hamidi *et al.* [1] combined the discrete wavelet transform (DWT), discrete cosine transform (DCT), and scale-invariant feature transform (SIFT) to support recovery after image processing and geometric changes. Their hybrid design illustrates why geometric distortion needs mechanisms beyond ordinary coefficient quantization.

Singh *et al.* [2] investigated blind adaptive watermarking with location and scaling choices intended to balance imperceptibility and robustness. Blind recovery is also a goal of the present application: decoding uses metadata embedded in the modified image rather than a separate original cover.

Li *et al.* [3] incorporated resampling detection and an auxiliary template to address scaling and cutting. In contrast, the MATLAB prototype discussed here uses a simpler fixed Haar block layout, a stored header, and repeated payload bits. The cited studies motivate the evaluation dimensions, but their reported performance is not transferred to this different implementation or dataset.

III. System Design

A. Interactive Workflow and Watermark Preparation

The interface provides Create, Extract, Details, and Evaluate views. The Create view loads a cover image and an image to hide. The latter is converted to grayscale, resized while preserving its proportions, and padded with white pixels to the user-selected square dimensions. Consequently, the recovery target is the *prepared* square, not the original watermark input. In the example, a 96×96 skyline image is converted into a 32×32 prepared grayscale watermark.

The MATLAB implementation, supporting Python files, test images, and project documentation are available in the project's GitHub repository [4].

Embedding strength s is an integer in the range 8–32. The interface maps this to a display percentage using $(s-8)/(32-8)$; thus, $s = 16$ corresponds to approximately 33% of the selectable range. The percentage is a slider position, not the fraction of image pixels changed. Classic, Balanced, and Durable modes control payload arrangement and repetition, as described next.

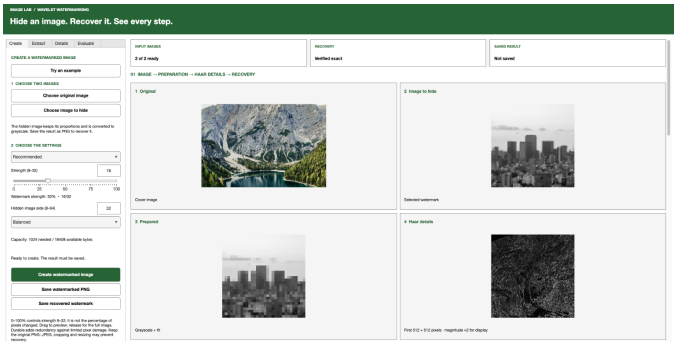


Fig. 1. The Create interface shows two inputs, strength controls, a prepared watermark, and a Haar detail preview.

B. Local Haar Transform and Embedding

Each image channel is divided into nonoverlapping 2×2 pixel blocks. An orthonormal representation of the local Haar transform is

$$\mathbf{c} = \mathbf{H}\mathbf{p}, \quad \mathbf{H} = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}, \tag{1}$$

where $\mathbf{p}$ contains four input samples, and $\mathbf{c}$ contains one low-frequency and three detail components. The implementation embeds bits in two selected detail coefficients, then reconstructs pixels through the inverse transform. Writing into detail rather than average coefficients seeks to limit visually prominent changes, but does not guarantee identical behavior for every image texture.

A parity-constrained quantization representation of the bit-writing rule is

$$q = \operatorname{round}\left(\frac{c}{s}\right), \quad q_b = 2\operatorname{round}\left(\frac{q-b}{2}\right) + b, \quad c' = sq_b, \tag{2}$$

where c is the coefficient before embedding and $b \in \{0, 1\}$ is the bit. The decoder estimates $q' = \operatorname{round}(c'/s)$ and uses its parity to recover b. This describes the parity mechanism reported for the implementation; the final pixel-domain output can also be affected by rounding and clipping.

Classic mode writes payload blocks sequentially, while Balanced and Durable modes distribute them across the cover. Classic and Balanced repeat each payload bit three times; Durable repeats it five times. The non-Classic modes additionally reduce local strength in relatively quiet blocks, identified when the magnitude of the fourth Haar coefficient is below 16. At the demonstrated global strength of 16, the reported quiet-block strength is 12.

C. Header, Capacity, and Verification

A repeated 24-byte header stores the decoding settings, including watermark dimensions, strength, and payload length. The header occupies $24 \times 8 \times 3 = 576$ coefficient slots. Each 2×2 cover block contributes two selected coefficient slots per encoded channel. For cover dimensions $H_c \times W_c$ and payload repetition factor r, the displayed payload capacity in bytes is

$$C = \left\lfloor \frac{2\lfloor H_c/2 \rfloor \lfloor W_c/2 \rfloor - 576}{8r} \right\rfloor. \tag{3}$$

Color-channel observations may support majority voting, but this payload-capacity calculation does not multiply the number of slots by three.

For a 1152×768 cover, there are $221{,}184$ two-by-two blocks and $442{,}368$ candidate slots. With $r = 3$, (3) yields $C = 18{,}408$ bytes. A 32×32 grayscale payload contains $1{,}024$ bytes or $8{,}192$ bits. Its threefold repetition uses $24{,}576$ slots; including the header gives $25{,}152$ occupied slots, which fit within the cover.

Extraction first validates the embedded header, then locates the payload using its stored settings. Repeated coefficient observations vote on individual bits; RGB channel observations provide further votes when supported. The bitstream is converted into grayscale bytes and checked against a stored payload checksum. The software distinguishes verified exact recovery from checksum failure, unreadable header, incomplete payload, and (where supported) approximate output. A visually recognizable preview by itself is not treated as proof of exact recovery.

Figure 2 presents the original MATLAB workflow without changing its structure. The left branch starts with loading and preparing the images, checking cover capacity, performing Haar-based embedding, and verifying recovery before saving. The right branch starts with a previously saved watermarked PNG, reads the embedded header, and recovers the grayscale watermark. Both branches lead to the optional saving of the recovered watermark. The diagram preserves its original decision points, processing order, and verification steps.

IV. Evaluation Method

A. Cover-Image Fidelity

Image quality is measured using the original cover X and its watermarked counterpart Y. For N samples across the evaluated image channels, mean squared error and PSNR are

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(X_i - Y_i)^2, \tag{4}$$

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{255^2}{\mathrm{MSE}}\right) \ \mathrm{dB}. \tag{5}$$

The application also computes a luminance-based SSIM variant on nonoverlapping 8×8 blocks. For each block, the standard structural form is

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \tag{6}$$

where μ denotes block mean, σ^2 variance, σ_{xy} covariance, and C_1, C_2 stabilizing constants. This program's *block luminance SSIM* must be labeled as such rather than assumed

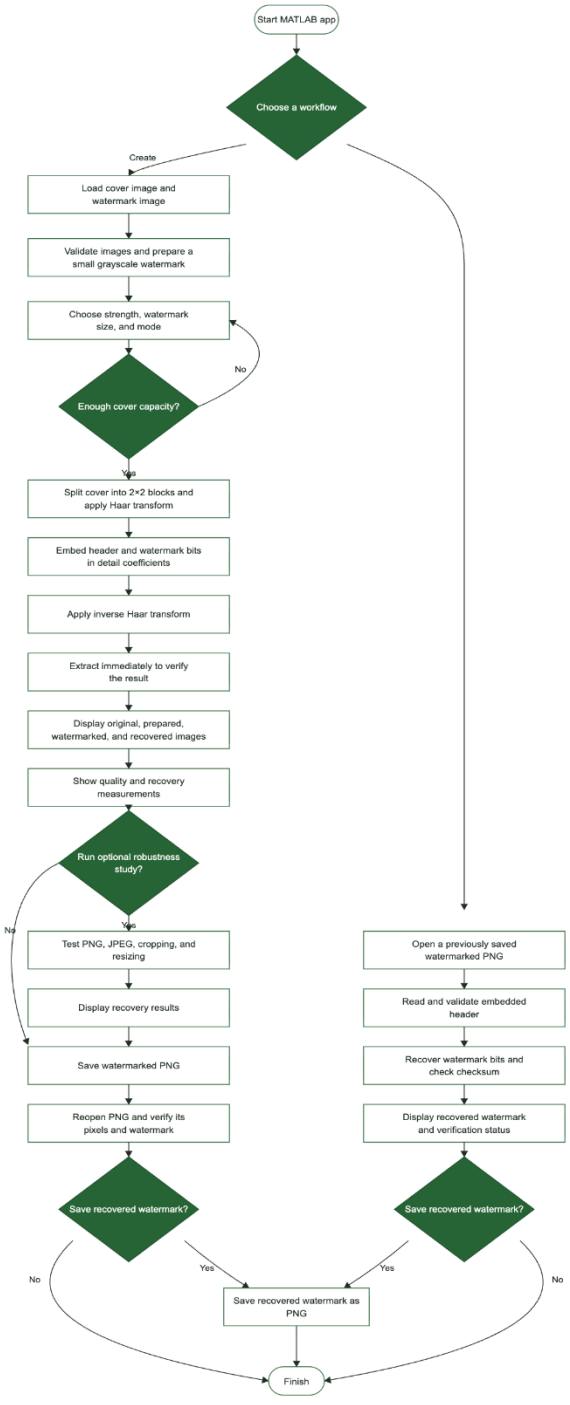


Fig. 2. Embedding and blind extraction workflow, including immediate verification, stored-header decoding, and checksum validation.

numerically identical to every standard SSIM software implementation.

B. Recovery Fidelity

Let W denote the *prepared* watermark, and $\widehat{W}$ the recovered grayscale image. For payloads of B bits, the bit error rate is

$$\mathrm{BER} = \frac{1}{B}\sum_{i=1}^{B} \mathbb{1}(w_i \neq \hat{w}_i) \times 100\%. \tag{7}$$

Normalized correlation is computed using a normalized pixel dot product:

$$\mathrm{NCC} = \frac{\sum_i W_i \widehat{W}_i}{\sqrt{\left(\sum_i W_i^2\right)\left(\sum_i \widehat{W}_i^2\right)}}. \tag{8}$$

BER checks exact bit agreement when comparable recovered bytes are available. A valid checksum offers a separate payload-integrity check, although it is not cryptographic authentication. When extraction fails to produce a comparable watermark, BER and NCC are unavailable rather than automatically equal to zero.

C. Robustness Study Design

The optional Evaluate function defines a factorial study with three modes, three strength values (8, 16, and 32), and three watermark sizes, producing 27 candidate settings. The study tests eight image conditions: unchanged PNG, JPEG quality 95, 75, and 50, two crop levels, and two resizing factors (75% and 50%). Settings without sufficient cover capacity are excluded. The working image is fit within a maximum width or height of 512 pixels. For attack type a, the exact recovery rate is

$$R_a = \frac{N_{\text{verified exact},a}}{N_{\text{tested},a}} \times 100\%. \tag{9}$$

A verified exact result requires a valid payload checksum and pixel-identical recovered prepared watermark. This strict definition avoids counting approximate but unverified images as successes.

V. Experimental Observations

A. Unchanged Watermark Recovery

The documented demonstration uses a 1152×768 mountain-and-lake cover and a 96×96 city-skyline watermark input. The selected watermark is prepared as a 32×32 grayscale square. At strength $s = 16$ and Balanced mode, the application reports *verified exact* extraction of the prepared watermark. The Details view reports the measurements in Table I. It also states that the current result has *not been saved*. These measurements therefore describe the observed in-memory example, not a separately documented end-user save-and-reopen operation.

Zero BER means no mismatches across 8,192 prepared bits. NCC of 1.00000 agrees with the exact recovered image. Equation (5) converts 44.29 dB PSNR to approximately 2.42 squared 8-bit intensity units of MSE per sample. The 0.9966 block SSIM indicates high measured similarity, without implying a universal pass threshold.

TABLE I
DOCUMENTED UNCHANGED-IMAGE RESULT

Measurement	Observed value
Cover size	1152×768 pixels
Original watermark input	96×96 pixels
Prepared/recovered watermark	32×32 pixels
Embedding mode and strength	Balanced; $s = 16$
Required payload	1,024 bytes
Available payload capacity	18,408 bytes
Cover PSNR	44.29 dB
Cover block-luminance SSIM	0.9966
Recovered payload BER	0.000%
Watermark NCC	1.00000
Recovery/checksum state	Verified exact
Saved state in Details view	Not saved

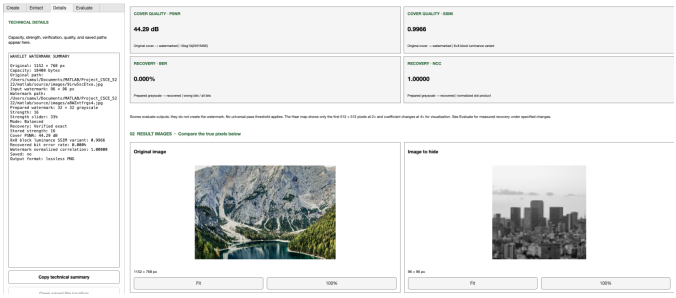


Fig. 3. Observed fidelity and recovery scores for the unchanged watermarked image.

B. Reported Robustness-Study Evidence

The Evaluate view uses a separate 512×342 working cover. Nine of its 27 settings are skipped for insufficient capacity, leaving 18 usable settings. At eight image changes per setting, there are 144 evaluated condition instances in the run described by the project report. One visible summary row contains six tests and a 100% exact recovery rate for unchanged PNG under the fitting Classic-mode settings. This supports unchanged PNG recovery in that study row, not a general claim that all other modes or attacks recover perfectly.

A visible study row reports 48.0198 dB PSNR and 0.9977 block SSIM for a different working cover. These are not substitutes for the main example's scores. Repeated quality values across attack rows may come from one embedding measured before the attacks. The difference preview is magnified fourfold for display, not error estimation.

Some additional rows show 0% exact recovery, but their attack labels are truncated. These cannot be attributed reliably to specific JPEG, crop, or resize conditions. Possible failure statuses include an unreadable header, incomplete payload, checksum failure, or approximate Durable-mode recovery.

VI. DISCUSSION AND LIMITATIONS

The unchanged run verifies the prepared 32×32 watermark, not the original 96×96 skyline. The main Details screen reports an unsaved result, while the separate study records six exact unchanged-PNG cases. Capacity depends on cover size, payload length, and repetition: the observed 1,024-byte payload

TABLE II
ROBUSTNESS EVIDENCE AND ITS SCOPE

Visible evidence	Supported interpretation
27 candidate settings; 9 skipped	18 capacity-fitting settings
8 changes per fitting setting	144 evaluated cases
6 unchanged-PNG tests; 100% exact	6/6 exact in the visible row
Other 0% rows with clipped labels	Attack-specific cause unknown
PSNR 48.0198 dB; SSIM 0.9977 in a study row	Different cover/configuration than Table I

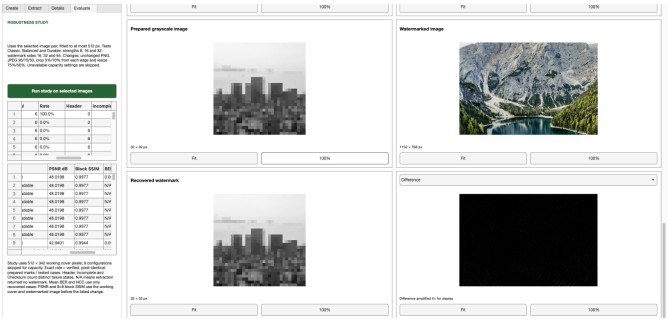


Fig. 4. Robustness-study settings and capacity-filtered configuration counts.

fits within 18,408 available bytes, but nine smaller-cover study settings exceeded capacity. Stronger quantization may improve coefficient separation at the cost of cover fidelity; controlled comparisons are needed to measure the tradeoff.

Cropping and resizing can disrupt fixed Haar block coordinates. Unlike feature-matching [1] and geometry-correction [3] approaches, this prototype relies on fixed locations and a stored header. JPEG compression can likewise perturb coefficients. Because several study labels are clipped, attack-specific success rates cannot be established from the available screenshots. Further evaluation should export complete labeled results, repeat tests across cover textures, and document a user save-and-reopen cycle. A checksum verifies recovered-byte consistency but does not prove ownership or secure authentication.

VII. CONCLUSION

The MATLAB Haar-wavelet application embeds, extracts, and evaluates a prepared grayscale watermark. Balanced mode at strength 16 recovers the demonstrated 32×32 payload exactly, with 44.29 dB PSNR, 0.9966 block SSIM, 0% BER, NCC of 1.00000, and a verified checksum. The study supports limited unchanged-PNG evidence but not universal robustness to image changes.

REFERENCES

[1] M. Hamidi, M. El Haziti, H. Cherifi, and M. El Hassouni, "A hybrid robust image watermarking method based on DWT-DCT and SIFT for copyright protection," *Journal of Imaging*, vol. 7, no. 10, Art. no. 218, 2021, doi: 10.3390/jimaging7100218.

[2] P. Singh, K. J. Devi, H. K. Thakkar, and J. Santamaría, "Blind and secured adaptive digital image watermarking approach for high imperceptibility and robustness," *Entropy*, vol. 23, no. 12, Art. no. 1650, 2021, doi: 10.3390/e23121650.

[3] H.-L. Li, X.-Q. Zhang, Z.-H. Wang, Z.-M. Lu, and J.-L. Cui, "Resampling-detection-network-based robust image watermarking against scaling and cutting," *Sensors*, vol. 23, no. 19, Art. no. 8195, 2023, doi: 10.3390/s23198195.

[4] sazzeb, "Project_CSCE_5222: Haar-wavelet image watermarking implementation," GitHub repository. [Online]. Available: https://github.com/sazzeb/Hough-Transform/tree/main/Project_CSCE_5222. Accessed: Nov. 10, 2026.